

Projeto final - SOE

Esteira automatizada

Felipe de Castro Quixabeira
Faculdade de Ciências e Tecnologia
Universidade de Brasília

Paulo Caleb Fernandes da Silva
Faculdade de Ciências e Tecnologia
Universidade de Brasília

Resumo—Este trabalho apresenta o projeto e a implementação de um sistema embarcado para o controle de uma esteira automatizada destinada à separação de itens de pequeno porte. O sistema proposto é capaz de transportar objetos com restrições de massa e volume, realizando a identificação e o direcionamento em tempo real com base em critérios previamente definidos. A arquitetura integra uma unidade de controle de baixo nível baseada em microcontrolador, sensores para detecção e classificação de objetos e atuadores responsáveis pela separação em dois pontos de saída, além de um *Single Board Computer (SBC)* dedicado ao processamento de tarefas de alto nível em ambiente Linux embarcado.

I. INTRODUÇÃO

Sistemas de logística interna em centros de distribuição modernos (Fig. 1) dependem fortemente de linhas automatizadas para transporte e separação de itens, permitindo maior eficiência na preparação de pedidos e redução da intervenção humana. Empresas como Amazon e Mercado Livre utilizam sistemas altamente integrados, compostos por esteiras, sensores e robôs, capazes de identificar, rastrear e direcionar produtos em tempo real ao longo de múltiplos estágios do processo logístico [1], [2], [3], [4], [5]. A amplitude desse ecossistema pode ser observada em diversas referências audiovisuais [6], [7], [8], [9], e a tendência é que haja ampliação desses sistemas robotizados para suprir a crescente demanda por entregas e rastreamentos [5]. Nesse contexto, módulos de esteiras desempenham papel fundamental como unidades básicas de movimentação e roteamento de itens dentro de arquiteturas modulares e escaláveis.



Figura 1. Centro de distribuição do Mercado Livre em Cajamar, São Paulo.

O problema central abordado neste trabalho consiste no desenvolvimento de um módulo de esteira automatizada capaz

de reproduzir, em escala reduzida, o comportamento esperado de um sistema de separação de itens em uma linha de produção. Considera-se um fluxo contínuo de objetos, sendo necessário detectar sua presença, identificar o destino de cada objeto por meio da leitura de um código QR e direcioná-los corretamente para diferentes saídas. Esse problema envolve desafios típicos de sistemas embarcados, como controle em tempo real, integração entre sensores e atuadores, comunicação com sistemas externos e divisão eficiente de tarefas entre diferentes níveis de processamento.

No estado da arte, soluções industriais empregam arquiteturas distribuídas e altamente escaláveis, nas quais o processamento é dividido entre controladores de baixo nível, responsáveis pelo gerenciamento do sistema mecatrônico em tempo real, e sistemas de alto nível, responsáveis por visão computacional, tomada de decisão e integração com sistemas de gestão. No meio acadêmico, diversos trabalhos exploram sistemas de separação automatizada baseados em visão computacional, identificação por código de barras ou QR code e controle embarcado de esteiras, frequentemente utilizando plataformas como microcontroladores e computadores de placa única (*Single Board Computers* — SBCs) (Fig. 2) para prototipagem e validação experimental. Entretanto, muitas dessas abordagens focam isoladamente na identificação ou no controle, não explorando de forma integrada a comunicação eficiente entre camadas de processamento com restrições reais de *hardware*. Diante desse cenário, torna-se relevante o desenvolvimento de um protótipo acadêmico que permita explorar, de forma controlada, os principais desafios envolvidos na integração entre controle embarcado e processamento de alto nível, sem a complexidade e o custo associados a sistemas industriais completos.

Dessa forma, o presente trabalho propõe o desenvolvimento de um sistema embarcado para controle de um módulo de esteira automatizada em escala reduzida com uma entrada e duas saídas, capaz de realizar a separação de itens com base em decisões externas previamente associadas a um código QR de identificação do objeto. A arquitetura adotada diferencia-se por utilizar uma abordagem de processamento distribuído, na qual uma unidade de controle baseada em microcontrolador é responsável pelas tarefas de tempo real, enquanto um *Single Board Computer* executando Linux embarcado realiza tarefas de mais alto nível, como captura de imagem, leitura de código QR, definição de destino e supervisão do sistema.

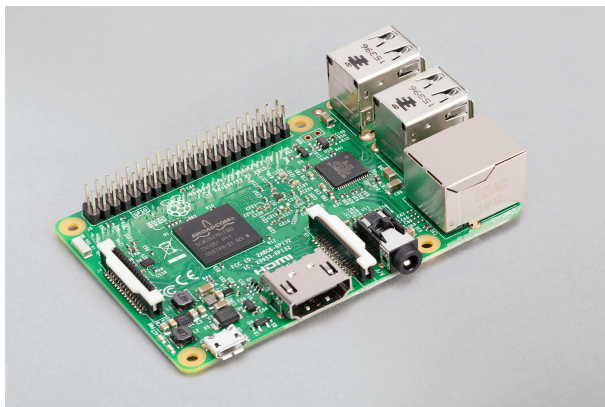


Figura 2. Raspberry Pi 3 Modelo B.

O sistema proposto tem como objetivo principal implementar um módulo funcional de separação de itens, capaz de detectar objetos, posicioná-los em uma zona de leitura, solicitar a identificação ao servidor Linux e executar o desvio para uma entre duas saídas possíveis. Para isso, são estabelecidos requisitos como a operação com objetos de pequeno porte (massa de até 250g, volume máximo de 125 cm³ e altura entre 3 cm e 5 cm), o processamento sequencial de itens, a comunicação confiável entre o módulo embarcado e o servidor supervisor, e a execução determinística das ações de controle.

Como resultado esperado, busca-se a validação experimental de um protótipo funcional que demonstre a viabilidade da integração entre controle embarcado em tempo real e processamento de alto nível em ambiente Linux embarcado. A solução proposta também se destaca por seu caráter modular e didático, podendo servir como base para expansão em sistemas maiores de automação logística. Os arquivos do projeto estão disponíveis em repositório público no GitHub [10].

II. SOLUÇÃO PROPOSTA

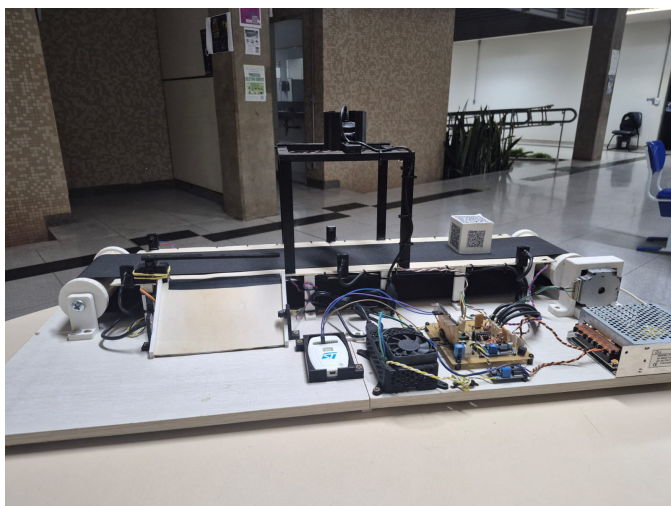


Figura 3. Foto da esteira

A. Descrição de hardware

O sistema é composto por uma arquitetura híbrida que divide as responsabilidades entre um microprocessador Broadcom BCM2837, hospedado na placa Raspberry Pi 3B e responsável pelas tarefas de alto nível, e um microcontrolador STM32G070, dedicado ao controle em tempo real dos dispositivos físicos da esteira. O diagrama de blocos da Fig. 4 ilustra de forma simplificada a arquitetura do sistema.

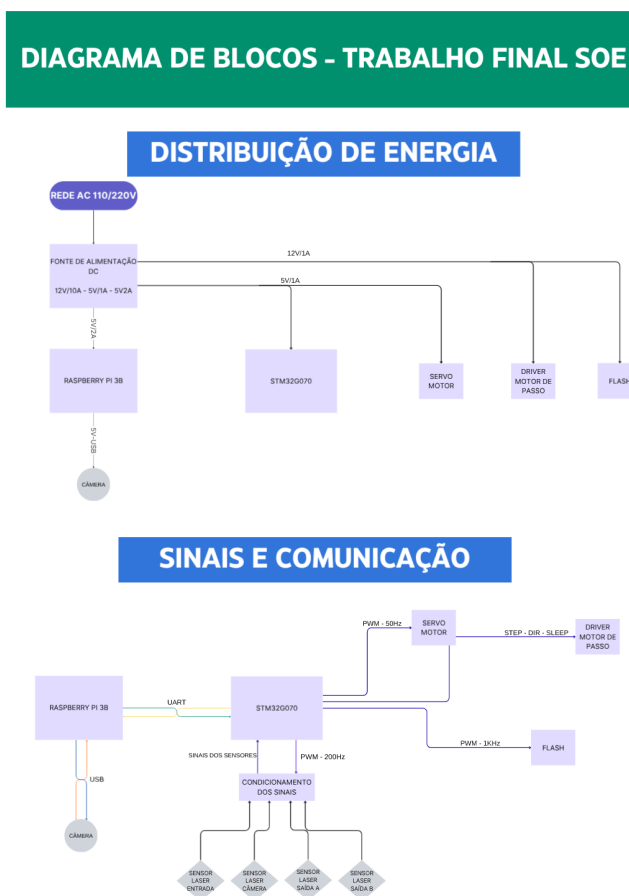


Figura 4. Diagrama de blocos do hardware.

A placa Raspberry Pi 3B foi escolhida devido à ampla documentação disponível, à acessibilidade ao *hardware*, uma vez que um dos integrantes da dupla já dispunha de uma unidade em funcionamento, e ao suporte nativo ao sistema operacional Linux, requisito para a execução da interface gráfica e dos algoritmos de visão computacional desenvolvidos neste projeto.

A câmera utilizada para a leitura e processamento de imagem é uma *webcam* genérica da marca Multilaser, com interface USB. A escolha foi motivada pela ausência de necessidade de *drivers* proprietários para o correto funcionamento com o Linux embarcado e pela disponibilidade imediata para o projeto.

O microcontrolador STM32G070 foi escolhido por sua variedade de periféricos adequados à aplicação, como comunicação UART, *timers* de uso geral e recurso de DMA (*Direct Memory Access*), que permite processar informações obtidas pelos periféricos sem ocupar a CPU. Além dessas características, o STM32G070 opera a uma frequência de CPU de 64 MHz e é suportado pelo ambiente de desenvolvimento integrado STM32CubeIDE, da STMicroelectronics, referência mundial na fabricação e suporte de microcontroladores para a indústria. O esquemático de conexão do microcontrolador pode ser observado na Fig. 5. Os *timers* são utilizados da seguinte forma:

- **TIM15:** gera o sinal PWM de frequência fixa (200 Hz, *duty cycle* de 50 %) utilizado para modular os lasers dos sensores;
- **TIM3:** configurado em modo *Input Capture* com DMA, realiza a medição de frequência dos quatro sensores laser simultaneamente;
- **TIM16:** gera o sinal PWM de 50 Hz com *duty cycle* variável para o controle do servo motor;
- **TIM17:** gera o sinal PWM de 1 KHz com *duty cycle* variável para o controle da luminária;
- **TIM6:** opera a 1 MHz e dispara interrupções periódicas para o controle preciso dos pulsos enviados ao *driver* do motor de passo.

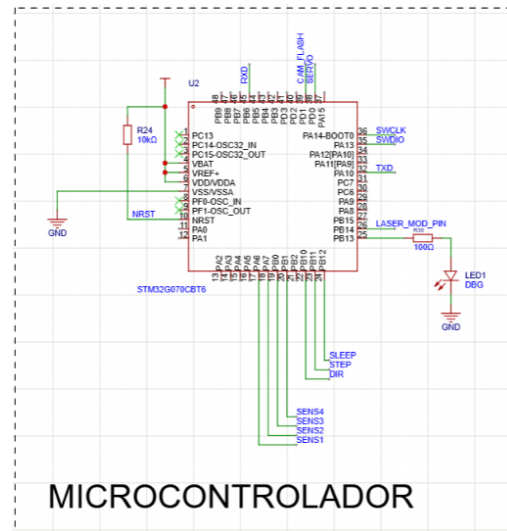


Figura 5. Esquemático do microcontrolador STM32G070.

A interface de programação e depuração do STM32G070 é realizada via *Serial Wire Debug* (SWD), conectada ao ST-Link V2 conforme o esquemático da Fig. 6. Os sinais SWCLK, SWDIO e NRST permitem gravar o *firmware* e depurar o sistema em tempo real diretamente pelo STM32CubeIDE.

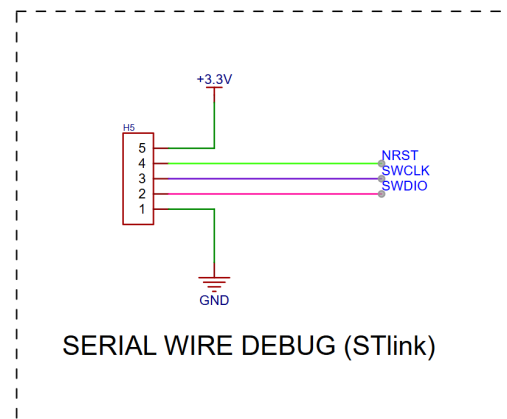


Figura 6. Esquemático da interface *Serial Wire Debug* (ST-Link).

A comunicação serial entre o STM32G070 e o Raspberry Pi 3B é realizada via UART, convertida para USB por meio de um módulo conversor dedicado, cujo esquemático é apresentado na Fig. 7. Esse conversor permite que o Raspberry Pi acesse a porta serial do microcontrolador como um dispositivo `/dev/ttyUSB`, sem necessidade de adaptação de nível lógico adicional, uma vez que tanto o STM32 quanto o conversor operam em 3,3 V.

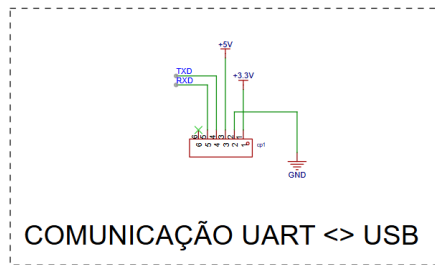


Figura 7. Esquemático do conversor UART ↔ USB.

A alimentação do sistema é gerenciada por dois estágios de regulação, conforme o esquemático da Fig. 8. A tensão de entrada de 12 V, proveniente de uma fonte externa, é convertida para 5 V pelo módulo *buck* LM2596S, protegido por um diodo 6A10 contra inversão de polaridade. O regulador linear AMS1117-3.3 deriva os 3,3 V necessários para a alimentação do STM32G070 e dos circuitos de sinal a partir dos 5 V. Capacitores de desacoplamento de 100 nF estão presentes na entrada e saída do AMS1117-3.3 para rejeição de ruído de alta frequência.

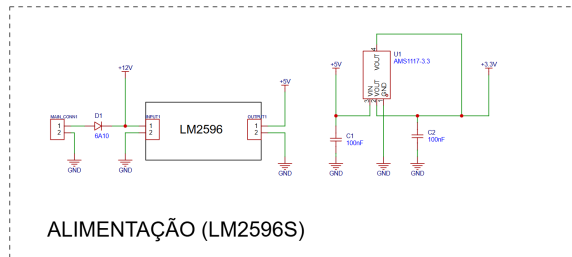


Figura 8. Esquemático do circuito de alimentação (LM2596S + AMS1117-3.3).

Como sistema motor do tapete da esteira, foi utilizado um motor de passo NEMA 22 controlado pelo *driver* A4988, conforme o esquemático da Fig. 9. Os pinos STEP, DIR e SLEEP são utilizados, respectivamente, para avançar um passo, definir a direção de rotação e habilitar ou desabilitar o *driver* (liberando ou travando o eixo). Esses componentes foram escolhidos por sua disponibilidade e pela baixa complexidade operacional do *driver*.

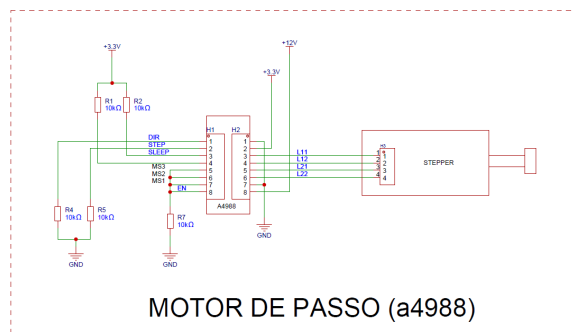


Figura 9. Esquemático do motor de passo e do *driver* A4988.

Como atuador da cancela, foi utilizado o servo motor ES08MA (Fig. 10), selecionado pela disponibilidade para o projeto e pelo torque superior ao do modelo SG90. O servo é controlado por PWM de 50 Hz, com largura de pulso variando entre 600 μ s (0°) e 2300 μ s (180°), mapeada diretamente a partir do ângulo desejado pelo *firmware*.

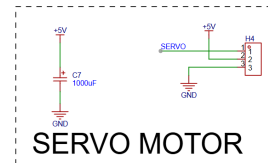


Figura 10. Esquemático do servo motor ES08MA.

Os sensores laser foram projetados para apresentar a maior simplicidade possível e para que o circuito eletrônico fosse viável de fabricação artesanal em *breakout boards* com solda manual. O princípio de funcionamento é baseado em detecção síncrona por ausência de portadora: o laser modula seu feixe na frequência gerada pelo TIM15 (200 Hz). O feixe incide sobre um LDR que, após condicionamento de sinal com amplificadores operacionais (LM324), entrega ao STM32G070 um sinal digital pulsado na faixa de 0 a 3,3 V com frequência idêntica à portadora. Quando um objeto interrompe o feixe, a ausência do sinal pulsado, interpretada pelo *firmware* como nível lógico alto permanente na entrada de captura, é identificada como presença de objeto à frente do sensor. O circuito conta com quatro instâncias desse sensor, dispostas em posições estratégicas ao longo da esteira: entrada da esteira (S1), zona de leitura do QR (S2), saída da Rota A (S3) e saída da Rota B (S4). O esquemático completo dos quatro sensores é apresentado na Fig. 11.

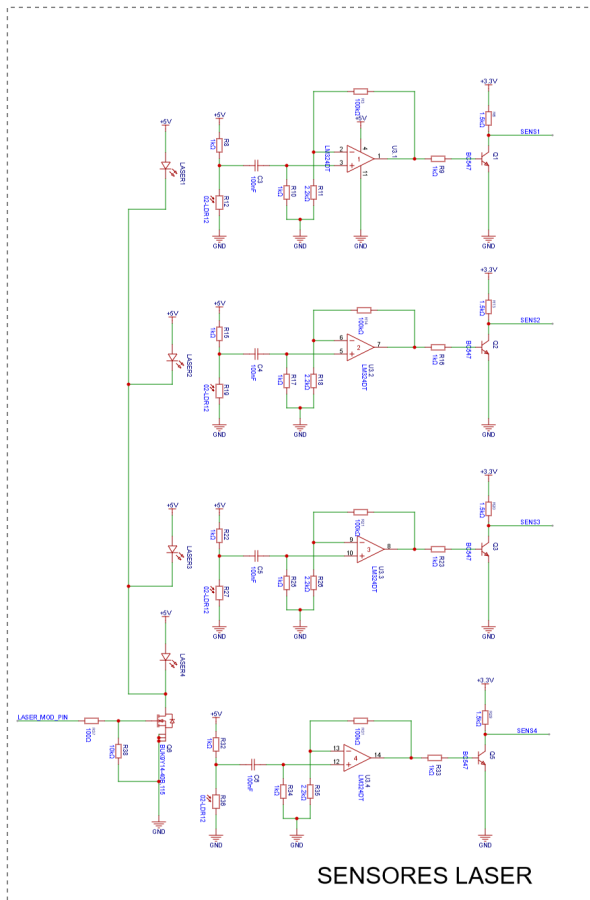


Figura 11. Esquemático dos quatro sensores laser com condicionamento de sinal (LDR + LM324).

Para auxiliar a visualização do código QR pela câmera, o sistema conta com uma luminária composta por seis LEDs de uma fita LED 4000 K. A temperatura de cor foi escolhida por apresentar maior similaridade com a luz ambiente, o que resultou em melhor qualidade de captura da câmera em comparação com fitas de 6500 K. O acionamento da luminária é realizado por PWM controlado pelo canal 1 do TIM17 do STM32G070, por meio de um MOSFET BUK9Y14 em configuração de chave, como ilustrado no esquemático da Fig. 12.

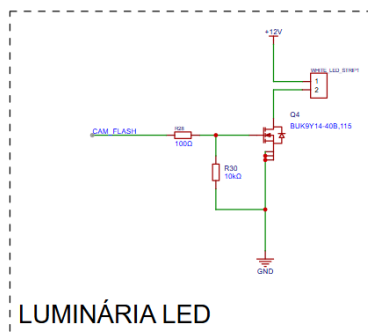


Figura 12. Esquemático do circuito de acionamento da luminária LED.

O esquemático completo da placa de circuito montada para o projeto encontra-se disponível no repositório do projeto [10].

Para a montagem completa do sistema, foi elaborada a lista de materiais apresentada nas Tabelas I e II, mais informações, incluindo os arquivos geratriz e .STL das peças podem ser encontrados no GitHub [10].

Tabela I
LISTA DE MATERIAIS — ELETRÔNICA.

Componente	Preço unit. (R\$)	Qtd.
Raspberry Pi 3B	—	1
Webcam genérica	—	1
ST-Link V2	—	1
STM32G070	5,50	1
Driver A4988	18,00	1
AMS1117-3.3	1,00	1
Módulo Buck LM2596S	15,00	1
AmpOp LM324	3,50	1
Diodo 1N4007	0,50	1
Capacitor eletrolítico 1000 μ F	0,50	1
Motor de passo NEMA 22	60,00	1
Servo motor ES08MA	25,00	1
MOSFET BUK9Y14	1,00	2
Transistor BJT BC547B	0,20	4
Resistor 100 Ω	0,10	3
Resistor 1 k Ω	0,10	12
Resistor 1k5 Ω	0,10	4
Resistor 2k2 Ω	0,10	4
Resistor 10 k Ω	0,10	8
Resistor 100 k Ω	0,10	4
LDR 10 k Ω	0,50	4
Laser com ajuste focal	1,00	4
Capacitor 100 nF	0,10	6
Fio coaxial (sensores LDR)	10,00	2 m
Fio 22 AWG vermelho (lasers)	1,50	2 m
Fio 22 AWG azul (lasers)	1,50	2 m
Fio 22 AWG preto (servo)	1,50	1 m
Fio 22 AWG branco (servo)	1,50	1 m
Fio 22 AWG amarelo (servo)	1,50	1 m
Conector Molex KK macho 2 terminais	0,10	8
Conector Molex KK macho 3 terminais	0,15	2
Conector Molex KK fêmea 2 terminais	0,10	8
Conector Molex KK fêmea 3 terminais	0,15	2
Total (itens com preço definido)	163,80	

Tabela II
LISTA DE MATERIAIS — MECÂNICA.

Componente	Preço unit. (R\$)	Qtd.
Perfil de alumínio 20×20 mm	—	—
Chapa de PVC (rampa saída B)	—	1
Tapete de PVC	—	1
Rolo motriz (impressão 3D)	—	1
Rolo bobó (impressão 3D)	—	1
Lateral esq. mesa esteira (imp. 3D)	—	1
Lateral dir. mesa esteira (imp. 3D)	—	1
Lateral esq. rampa (imp. 3D)	—	1
Lateral dir. rampa (imp. 3D)	—	1
Cancela servo (imp. 3D)	—	1
Suporte servo (imp. 3D)	—	1
Suporte ST-Link (imp. 3D)	—	1
Suporte laser c/ ajuste focal (imp. 3D)	—	3
Suporte LDR (imp. 3D)	—	3
Suporte laser rampa (imp. 3D)	—	1
Suporte LDR rampa (imp. 3D)	—	1
Suporte flash esteira (imp. 3D)	—	2
Suporte poste câmera (imp. 3D)	—	2
Suporte câmera esteira (imp. 3D)	—	1

B. Descrição de software

O *software* do sistema é dividido em dois níveis de processamento: o *firmware* embarcado no STM32G070, responsável pelo controle em tempo real dos atuadores e sensores, e o programa de alto nível executado no Raspberry Pi 3B, responsável pela supervisão do sistema, leitura de código QR e comunicação com o microcontrolador.

1) *Firmware do STM32G070*: O *firmware* foi desenvolvido em linguagem C utilizando os *drivers* HAL da STMicroelectronics no ambiente STM32CubeIDE. Sua estrutura principal é composta por uma rotina de inicialização, um laço principal (*loop* infinito) e uma Máquina de Estados Finitos (FSM).

a) *Rotina de inicialização*.: Antes de entrar no laço principal, o *firmware* executa uma rotina de bloqueio que aguarda simultaneamente duas condições: (i) todos os quatro sensores laser devem operar na frequência esperada de 200 Hz, e (ii) o Raspberry Pi deve enviar o byte de *handshake* SYS_RDY (0x10). Durante esse período, um LED de depuração indica o estado do sistema: pisca a 100 ms se os sensores ainda não estiverem prontos, e a 500 ms quando aguarda apenas o *handshake*. Ao receber o sinal do Raspberry Pi, o STM32G070 responde com SYS_INIT (0x01) e ingressa no laço principal.

b) *Leitura dos sensores laser*.: A leitura dos sensores é baseada em detecção síncrona por ausência de portadora. O TIM15 gera um PWM de 200 Hz para modular os quatro lasers simultaneamente. O TIM3, em modo *Input Capture* com DMA, mede a frequência do sinal retornado por cada LDR. Se a frequência capturada corresponde à portadora (200 Hz), o feixe está livre; se a captura retorna zero (ausência de sinal), há um objeto interrompendo o feixe. O laço principal chama `getSensorsReadings()` a cada intervalo definido, atualizando as variáveis de estado `SENSn_freq` e `SENSn_flag` para cada sensor $n \in \{1, 2, 3, 4\}$.

c) *Controle do motor de passo*.: O motor de passo NEMA 22 é acionado via TIM6, que opera a 1 MHz e gera interrupções periódicas para o controle preciso dos pulsos enviados ao pino STEP do *driver* A4988. O sentido de rotação é determinado pelo pino DIR, e o pino SLEEP controla o engajamento do eixo. Funções auxiliares como `stepperMove()`, `stepperSetSpeed()`, `stepperStopEngaged()` e `stepperStopDisengaged()` encapsulam o controle do motor, permitindo que a FSM abstraia os detalhes de temporização.

d) *Controle do servo motor*.: O servo ES08MA é controlado pelo TIM16, que gera um PWM de 50 Hz com *duty cycle* variável. A função `SetServoAngle(angle)` mapeia o ângulo desejado (em graus) para o valor de comparação do registrador do *timer*, com largura de pulso variando de 600 μ s (0°) a 2300 μ s (180°). No contexto da FSM, a cancela opera em dois estados: fechada (0°), posicionando os objetos para a Rota A, e aberta (45°), desviando-os para a Rota B.

e) *Máquina de Estados Finitos (FSM)*.: O fluxo de operação do sistema é controlado por uma FSM implementada

na função `FSM()`, chamada ciclicamente no laço principal. O diagrama de estados é apresentado na Fig. 13. Os estados são:

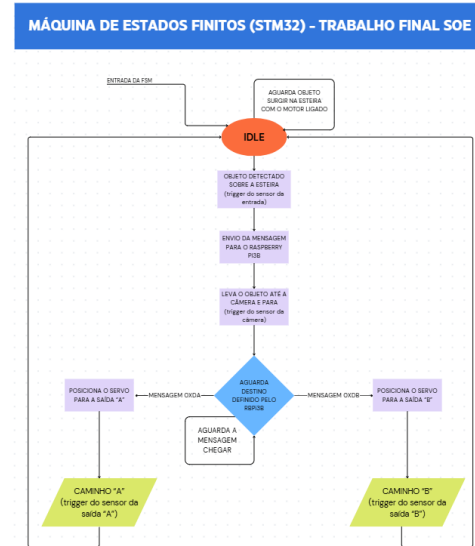


Figura 13. Diagrama de estados da FSM do *firmware* do STM32G070.

- **IDLE**: sistema aguardando objeto. Motor ligado, cancela fechada, *flash* apagado;
- **OBJ_DETECTED**: objeto detectado pelo sensor S1. Motor continua em movimento transportando o objeto até a zona de leitura;
- **CLASSIF_REQUEST**: objeto parado sob o sensor S2 (zona de leitura). Motor para. *Flash* aceso. STM32 envia `CLASS_REQUEST` (0xC0) ao Raspberry Pi e aguarda resposta de roteamento;
- **ROUTE_A**: Raspberry Pi enviou `ROUTE_A` (0xDA). Cancela permanece fechada. Motor reinicia. STM32 envia `ROUTE_A_FWRDNG` (0xFA) e aguarda detecção no sensor S3;
- **ROUTE_B**: Raspberry Pi enviou `ROUTE_B` (0xDB). Cancela abre (45°). Motor reinicia. STM32 envia `ROUTE_B_FWRDNG` (0xFB) e aguarda detecção no sensor S4;
- **DELIVERED**: sensor de saída detectou o objeto. Mensagem de entrega bem-sucedida enviada ao Raspberry Pi. Sistema retorna a IDLE.

f) *Modo de depuração (DEBUG)*.: O *firmware* implementa um modo de operação alternativo, ativado pelo comando `DEBUG_TOGGLE` (0xDD). Nesse modo, a FSM é suspensa e o STM32G070 passa a responder exclusivamente a comandos assíncronos enviados pelo Raspberry Pi, permitindo o acionamento individual de cada atuador (*flash*, cancela, motor de passo) para fins de diagnóstico e validação. O STM32 também envia espontaneamente a telemetria dos quatro sensores (`SENS_STATUS`, 0x55) sempre que o estado de algum sensor muda.

g) *Protocolo de comunicação UART*.: A comunicação entre o STM32G070 e o Raspberry Pi 3B segue um protocolo proprietário desenvolvido para este projeto, operando

a 115200bps (8N1 — oito bits de dados, sem paridade, um *stop bit*). O recebimento é gerenciado por interrupção (HAL_UART_Receive_IT), garantindo que nenhum byte seja perdido durante a execução da FSM. A estrutura dos quadros é definida conforme a Tabela III, e as principais mensagens do protocolo são apresentadas na Tabela IV.

Tabela III
ESTRUTURA DOS QUADROS UART.

Direção	Formato
RPi → STM32 (RX)	[0xAA] [CMD] ou [0xAA] [0xE5] [DATA]
STM32 → RPi (TX ok)	[0x90] [PAYLOAD]
STM32 → RPi (TX erro)	[0x91]

Tabela IV
MENSAGENS DO PROTOCOLO UART PROPRIETÁRIO.

Nome	Descrição	Valor	Dir.
<i>Handshake</i>			
SYS_RDY	RPi sinaliza pronto	0x10	RX
SYS_INIT	STM32 confirma	0x01	TX
<i>Mensagens da FSM</i>			
OBJ_DETECTED	Objeto detectado (S1)	0xA0	TX
CLSS_REQUEST	Solicita classificação	0xC0	TX
ROUTE_A	Ordena rota A	0xDA	RX
ROUTE_B	Ordena rota B	0xDB	RX
ROUTE_A_FWRDNG	Confirm. encaminh. A	0xFA	TX
ROUTE_B_FWRDNG	Confirm. encaminh. B	0xFB	TX
ROUTE_A_OK	Entrega concluída A	0xBA	TX
ROUTE_B_OK	Entrega concluída B	0xBB	TX
<i>Controle assíncrono (modo DEBUG)</i>			
LIGHT_EN	Liga luminária	0xE1	RX
LIGHT_DIS	Desliga luminária	0xD1	RX
GATE_OPEN	Abre cancela (45°)	0xE2	RX
GATE_CLOSE	Fecha cancela (0°)	0xD2	RX
STPR_EN	Engaja motor	0xE3	RX
STPR_DIS	Libera motor	0xD3	RX
STPR_FWD	Rotação: frente	0xE4	RX
STPR_BCK	Rotação: trás	0xD4	RX
STPR_TGT_STPS	Define <i>N</i> passos	0xE5	RX
SENS_STATUS	Telemetria sensores	0x55	TX

2) *Software de alto nível — Raspberry Pi 3B*: O programa de alto nível, denominado esteira, foi desenvolvido em linguagem C (com exceção do módulo de câmera, implementado em C++ por exigência da API OpenCV 4.x). É compilado a partir de seis módulos independentes coordenados pelo arquivo `main.c`, conforme a Tabela V.

Tabela V
MÓDULOS DO *software* ESTEIRA.

Arquivo	Responsabilidade
<code>main.c</code>	Entrada, argumentos CLI, <i>handshake</i> e seleção de modo
<code>serial.c</code>	Camada física UART via <i>termios</i> (POSIX)
<code>protocol.c</code>	Montagem e <i>parsing</i> de quadros binários
<code>fsm.c</code>	Modo autônomo: FSM + <i>thread</i> RX + câmera
<code>debug.c</code>	Modo manual: menu interativo + <i>thread</i> RX
<code>camera.cpp</code>	Captura de <i>frames</i> (OpenCV) e leitura de QR (ZBar)
<code>log.c</code>	Log com <i>timestamp</i> e cores ANSI por categoria

a) *Inicialização e handshake*.: Ao ser executado, o programa realiza o *handshake* com o STM32G070 via `do_handshake()`: envia [0xAA] [0x10] (SYS_RDY) e

aguarda [0x90] [0x01] (SYS_INIT) por até 10000 ms. Em caso de *timeout* ou CMD_ERR, o programa encerra com código de erro. Em caso de sucesso, apresenta o menu de seleção de modo.

b) *Módulo serial.c — camada física*.: Abre o dispositivo com O_RDWR | O_NOCTTY | O_NONBLOCK e configura a porta via *termios* para operação *raw* 8N1, sem eco e sem controle de fluxo. A função `serial_read_byte_timeout()` lê um byte com *timeout* via `select()`, permitindo que as *threads* RX verifiquem periodicamente a *flag* de encerramento sem bloquear indefinidamente.

c) *Módulo protocol.c — camada de protocolo*.: Implementa o *parser* de quadros RX como uma máquina de três estados alimentada byte a byte por `protocol_parse_byte()`: (i) **WAIT_STATUS** — aguarda 0x90 ou 0x91, descartando qualquer outro byte com registro de erro; (ii) **WAIT_CMD** — aguarda o byte de comando; se for 0x55, avança para o terceiro estado; (iii) **WAIT_DATA** — aguarda o byte de telemetria e entrega o quadro completo de 3 bytes.

d) *Módulo fsm.c — modo autônomo*.: Gerencia duas *threads* concorrentes: a *thread* RX, criada antes da inicialização da câmera para não perder eventos do STM32 durante os ~2s de abertura do OpenCV, e a *thread* principal, que executa a FSM e a leitura do QR de forma síncrona. A exclusão mútua entre *threads* é garantida por `pthread_mutex_t` sobre a variável `ctx.rx_event`. Os estados da FSM do lado do Raspberry Pi são apresentados na Tabela VI.

Tabela VI
ESTADOS DA FSM DO RASPBERRY PI (FSM.C).

Estado	Condição de transição / ação
PC_FSM_IDLE	Aguarda OBJ_DETECTED (0xA0)
PC_FSM_OBJ_DETECTED	Aguarda CLSS_REQUEST (0xC0)
PC_FSM_CLASSIFYING	Aciona câmera; envia rota ao STM32
PC_FSM_ROUTE_A	Aguarda ROUTE_A_OK (0xBA)
PC_FSM_ROUTE_B	Aguarda ROUTE_B_OK (0xBB)
PC_FSM_DELIVERED_x	Aguarda 2s e retorna a IDLE

e) *Módulo debug.c — modo manual*.: Apresenta um menu interativo no terminal lido via `select()` sobre STDIN_FILENO com *timeout* de 200 ms, evitando o bloqueio da *thread* RX. A Tabela VII e a Fig. 14 listam os comandos disponíveis. A *thread* RX exibe a telemetria dos sensores (SENS_STATUS, 0x55) conforme ilustrado na Fig. 15, atualizando os indicadores S1–S4 sempre que o estado de algum sensor muda.

f) *Módulo camera.cpp — captura e decodificação de QR*.: Único módulo em C++, com funções exportadas via `extern "C"` para compatibilidade com os demais módulos C. Captura *frames* em 640×480 px via `cv::VideoCapture`, converte para escala de cinza e entrega o *buffer* ao ZBar em formato Y800. O *parser* interno `parse_qr_text()` busca primeiro o campo *Destino*: no texto do QR e, em seguida, realiza varredura linear de tokens

```

File Edit Tabs Help
caleb@raspberrypi1:~/Desktop/Testes_em_C_TF_SOE/GUI_c_V1 $ ./esteira -m 2
+=====+
| ESTEIRA SEPARADORA -- Sistema de Controle |
| STM32G070 <-> Raspberry Pi 3B | Firmware v3 |
| SOE 2026 -- Interface CLI em C |
+=====+
17:17:27.331 [SYS ] Porta: /dev/ttyS0 | Baud: 115200 | Camera: 0 | QR
timeout: 30000 ms
17:17:27.331 [SYS ] Abrindo porta serial /dev/ttyS0 @ 115200 bps...
17:17:27.332 [SYS ] Serial OK: /dev/ttyS0 | fd=3 | 115200 bps | 8N1 | sem fl
ow control
17:17:27.332 [SYS ] Porta serial aberta: /dev/ttyS0 @ 115200 bps
17:17:27.332 [SYS ] Enviando SYS_RDY (0x10) -- aguardando SYS_INIT...
17:17:27.332 [UART TX] >> TX [0xAA][0x10] SYS_RDY (0x10)
17:17:27.332 [UART RX] << RX [0x90][0x01] SYS_INIT (0x01)
17:17:27.333 [SYS ] Handshake OK -- sistema inicializado.
17:17:27.333 [SYS ] Iniciando modo DEBUG...
17:17:27.333 [DBG ] Modo DEBUG iniciado.

===== MODO DEBUG =====
1 Flash ON
2 Flash OFF
3 Cancela ABRIR
4 Cancela FECHAR
5 Motor ENGAJAR
6 Motor LIVRE
7 Direcao FRENTE
8 Direcao TRAS
9 Enviar passos (solicita quantidade)
t Toggle modo DEBUG/FSM
r SW Reset STM32
q Sair do programa

Comando: 17:17:27.333 [SYS ] Thread RX (DEBUG) iniciada -- monitorando UART.
17:17:27.344 [UART RX] << RX [0x90][0x11] MODE_FSM (0x11)

```

Figura 14. Menu do modo DEBUG exibido no terminal (debug.c).

```

File Edit Tabs Help
5 Motor ENGAJAR
6 Motor LIVRE
17:21:20.908 [UART RX] << RX [0x90][0x11] MODE_FSM (0x11)
7 Direcao FRENTE
8 Direcao TRAS
9 Enviar passos (solicita quantidade)
t Toggle modo DEBUG/FSM
r SW Reset STM32
q Sair do programa

===== MODO DEBUG =====
1 Flash ON
2 Flash OFF
3 Cancela ABRIR
4 Cancela FECHAR
5 Motor ENGAJAR
6 Motor LIVRE
7 Direcao FRENTE
8 Direcao TRAS
9 Enviar passos (solicita quantidade)
t Toggle modo DEBUG/FSM
r SW Reset STM32
q Sair do programa

Comando: t
17:21:23.609 [DBG ] Toggle DEBUG/FSM
17:21:23.609 [UART TX] >> TX [0xAA][0xDD] DEBUG_TOGGLE (0xDD)

===== MODO DEBUG =====
1 Flash ON
2 Flash OFF
3 Cancela ABRIR
4 Cancela FECHAR
5 Motor ENGAJAR
6 Motor LIVRE
7 Direcao FRENTE
8 Direcao TRAS
9 Enviar passos (solicita quantidade)
t Toggle modo DEBUG/FSM
r SW Reset STM32
q Sair do programa

Comando: << RX [0x90][0xDD] DBG_TOGGLE eco (0xDD)
17:21:23.609 [UART RX] << RX [0x90][0x22] MODE_DEBUG (0x22)
17:21:24.095 [SENSORES] S1:[LIVRE] S2:[LIVRE] S3:[LIVRE] S4:[LIVRE]
17:21:31.217 [SENSORES] S1:[OBJETO] S2:[LIVRE] S3:[LIVRE] S4:[LIVRE]
17:21:32.152 [SENSORES] S1:[LIVRE] S2:[LIVRE] S3:[LIVRE] S4:[LIVRE]
17:21:33.086 [SENSORES] S1:[LIVRE] S2:[OBJETO] S3:[LIVRE] S4:[LIVRE]
17:21:33.943 [SENSORES] S1:[LIVRE] S2:[LIVRE] S3:[LIVRE] S4:[LIVRE]
17:21:35.119 [SENSORES] S1:[LIVRE] S2:[LIVRE] S3:[OBJETO] S4:[LIVRE]
17:21:36.042 [SENSORES] S1:[LIVRE] S2:[LIVRE] S3:[LIVRE] S4:[LIVRE]
17:21:36.988 [SENSORES] S1:[LIVRE] S2:[LIVRE] S3:[LIVRE] S4:[OBJETO]
17:21:37.977 [SENSORES] S1:[LIVRE] S2:[LIVRE] S3:[LIVRE] S4:[LIVRE]

```

Figura 15. Telemetria dos sensores S1–S4 em tempo real (SENS_STATUS, 0x55).

Tabela VII
COMANDOS DO MODO DEBUG (DEBUG.C).

Tecla	Ação	Quadro TX
1	Flash ON	[0xAA][0xE1]
2	Flash OFF	[0xAA][0xD1]
3	Cancela abrir	[0xAA][0xE2]
4	Cancela fechar	[0xAA][0xD2]
5	Motor engajar	[0xAA][0xE3]
6	Motor livre	[0xAA][0xD3]
7	Direção: frente	[0xAA][0xE4]
8	Direção: trás	[0xAA][0xD4]
9	Enviar <i>N</i> passos	[0xAA][0xE5] [<i>N</i>]
t	Toggle DEBUG/FSM	[0xAA][0xDD]
r	SW Reset STM32	[0xAA][0x33]
q	Encerrar	—

hexadecimais, garantindo compatibilidade com diferentes formatos. A correspondência é 0xDA → Rota A e 0xDB → Rota B.

g) *Módulo log.c — sistema de log.*: Todas as mensagens seguem o formato HH:MM:SS.mmm [CATEGORIA] mensagem, com *timestamp* gerado via `clock_gettime(CLOCK_REALTIME)` e categorias codificadas por cores ANSI ([UART TX], [UART RX], [FSM], [CAM], [DBG], [SYS], [ERR]).

h) *Gerador de códigos QR.*: Para a criação dos códigos QR utilizados nos testes, foi desenvolvido um script Python auxiliar (`gerar_qrcodes.py`) que lê um arquivo JSON com as informações das peças e gera as imagens correspondentes. Cada código QR codifica o nome da peça e o destino (endereço hexadecimal utilizado pela FSM) conforme o formato arbitrado pela dupla.

Todos os arquivos podem ser encontrados no repositório do projeto [10].

III. RESULTADOS EXPERIMENTAIS

Os experimentos descritos nesta seção foram realizados com o objetivo de validar o funcionamento de cada subsistema fundamental da esteira, bem como a integração entre o microcontrolador STM32G070 e o Raspberry Pi 3B. Para cada subsistema, descreve-se o procedimento adotado, os resultados obtidos e, quando pertinente, as divergências observadas e suas possíveis causas.

a) *Sensores laser.*: Os quatro sensores operam conforme especificado. A lógica de detecção síncrona por ausência de portadora foi validada com sucesso — ao interromper o feixe, as *flags* internas (SENS1_flag a SENS4_flag) são acionadas corretamente, permitindo as transições de estado da FSM. A mudança das flags pode ser observada na Figura 15, na seção de software. Além disso, os testes de hardware (Figura 16) mostram que os sensores funcionaram bem em testes de bancada, como ilustrado nas figuras 17 e 18, que mostram a transição do sinal entre laser sem interromper e interrompido.

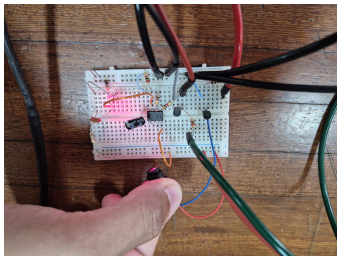


Figura 16. Circuito dos lasers montado em bancada

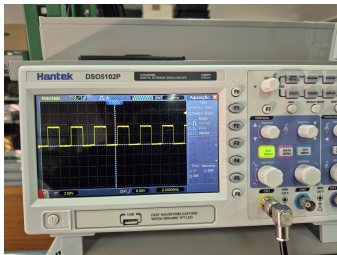


Figura 17. Laser sem interrupção

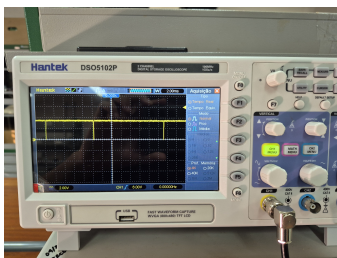


Figura 18. Laser interrompido

b) *Flash (luminária).*: O controle via PWM funciona corretamente, alterando a intensidade do brilho do *flash* no modo FSM e ativando e desativado em resposta aos comandos assíncronos recebidos via UART, sem instabilidades observadas.

c) *Motor de passo.*: O motor opera de forma estável em frequências mais baixas de acionamento. Os testes indicaram que o controle de velocidade por alteração do ARR do TIM6 funciona, porém com uma faixa operacional limitada — comportamento esperado dadas as características físicas do motor de passo (torque *versus* velocidade). Em razão disso, a velocidade de operação será mantida fixa no valor mínimo configurado ($\text{STEPPER_MIN_VEL} = 300 \text{ passos/s}$), que corresponde a um período de interrupção do TIM6 de aproximadamente 1,67 ms, calculado com base na configuração de *prescaler* de 63 e *clock* de 64 MHz.

d) *Servo motor.*: O servo motor ES08MA responde corretamente a qualquer valor de angulação dentro da faixa suportada (0° a 180°). Para a aplicação na esteira, os ângulos definidos no *firmware* — 0° para cancela fechada (Rota A) e 45° para cancela aberta (Rota B) — foram validados e apresentaram o posicionamento mecânico desejado.

e) *Dimensões do código QR.*: Observando o comportamento da câmera, foi possível verificar que um código QR com dimensões de $3 \times 3 \text{ cm}$ conseguiu ser lido com maior facilidade do que um código QR com dimensões de $5 \times 5 \text{ cm}$. Além disso, para auxiliar na leitura do código, foi implementada uma rotina de *fade-in/fade-out* do flash, garantindo que houvesse uma variação na luminosidade incidente no código QR. Essa variação auxiliou a leitura do código por permitir que o ajuste automático de saturação da *webcam* se adaptasse à variação de luz ambiente, permitindo que ao menos um dos quadros capturados pelo sensor estivesse suficientemente nítido para a correta interpretação do código pelo algoritmo.

f) *Instabilidades observadas na versão.*: Durante o desenvolvimento e validação do sistema, foram identificadas instabilidades que demandaram refinamentos ao longo do ciclo de implementação. No âmbito do *software*, três ocorrências foram tratadas: a interface de linha de comando desenvolvida em C apresentava comportamento inconsistente em determinadas sequências de operação, exigindo revisão da lógica de controle e do fluxo de execução; o procedimento de *reset* por software do STM32G070, acionado via comando `SW_RESET` pela interface, não respeitava o intervalo mínimo necessário para a reinicialização completa do microcontrolador, causando falha no handshake subsequente; por fim, foi identificada uma condição de corrida na etapa de inicialização do sistema, em que o Raspberry Pi 3B tentava estabelecer comunicação antes que os periféricos do STM32G070 estivessem devidamente configurados, resultando em perda do frame de handshake.

Como perspectiva para o próximo ponto de controle, no âmbito do *hardware*, prevê-se a incorporação de um *display* e de um botão físico à plataforma Raspberry Pi 3B. Tais recursos permitiriam ao operador interagir diretamente com o sistema embarcado sem depender de um terminal externo, tornando a operação mais autônoma e a interface com o usuário mais intuitiva, características desejáveis para uma versão de produto consolidada.

REFERÊNCIAS

- [1] Engenharia Híbrida, “Conheça o processo inovador de gestão de estoques da Amazon,” Disponível em <https://www.engenhariahibrida.com.br/post/conheca-processo-inovador-gestao-de-estoques-amazon> (30/03/2026).
- [2] Equipe TOTVS, “Logística do Mercado Livre: entenda os pilares da eficiência de distribuição da empresa,” Disponível em <https://www.totvs.com/blog/gestao-logistica/logistica-mercado-livre/> (30/03/2026), 12 2023.
- [3] —, “Logística da Amazon: entenda os pilares da empresa líder do varejo mundial,” Disponível em <https://www.totvs.com/blog/gestao-logistica/logistica-amazon/> (30/03/2026), 04 2024.
- [4] Mercado Livre, “Mercado Envios Full,” Disponível em <https://www.mercadolivre.com.br/l/envios-full> (30/03/2026).
- [5] Amazon Staff, “Amazon unveils the next generation of fulfillment centers powered by AI and 10 times more robotics,” Disponível em <https://www.aboutamazon.com/news/operations/amazon-fulfillment-center-robotics-ai> (30/03/2026).
- [6] YouTube, “Vídeo – referência de automação logística,” Disponível em <https://www.youtube.com/watch?v=cRC4iktG-kE> (30/03/2026).
- [7] —, “Vídeo – escaneamento e automação de itens em esteira,” Disponível em <https://youtu.be/iumMexYY1hg?t=111> (30/03/2026).
- [8] —, “Vídeo – sistema de separação e desvio de itens,” Disponível em <https://www.youtube.com/watch?v=7AHLNGUSeGQ&t=8s> (30/03/2026).

- [9] Instagram, “Reel – demonstração de sistema automatizado de esteira,” Disponível em <https://www.instagram.com/reels/DQKF8Hbja6q/> (30/03/2026).
- [10] Caleb P. e Castro F., “Trabalho final SOE 2026.1,” Disponível em https://github.com/pauloCaleb/Trabalho_Final_SOE_2026.1_Esteira.git (30/03/2026).

Os documentos a seguir estão disponíveis no repositório do projeto [10], servem como documentação local do repositório e estão sendo apresentados como apêndices a esse trabalho:

APÊNDICE

- **Apêndice A — Descrição das Peças** — descrição de cada peça mecânica fabricada por impressão 3D para a esteira, incluindo função e quantidade.
- **Apêndice B — Fechamento do Escopo** — documento que concentra de forma sucinta o contexto de aplicação, os objetivos, as limitações e as referências bibliográficas do projeto.
- **Apêndice C — Esquemático da placa com STM32G070**

Descrição das Peças Mecânicas – Esteira Separadora SOE

Peças do Subsistema Estrutural

Autor: Paulo Caleb Fernandes da Silva

Wood_stitches_Esteira_Rolante_SOE.catPART

Quantidade: 3 unidades

Peças plásticas retangulares com dois furos espaçados para fixar duas partes de madeira que compõem a base da esteira. Funcionam como "pontos" que seguram uma madeira na outra e conferem rigidez à base.

Apoio_Base_Esteira_Rolante_SOE.catPART

Quantidade: 8 unidades

Bases de apoio retangulares com quatro furos para parafusos e um ressalto para servir de pé para a esteira. São utilizadas para duas funções simultaneamente: complementam a costura das peças de madeira e servem de espaçadores da base em relação ao solo.

Suporte_Motor_Esteira_Rolante_SOE.catPART

Quantidade: 1 unidade

Base de fixação do motor de passo para garantir estabilidade mecânica do motor NEMA 22 e alinhá-lo com o suporte do rolo.

Trava_Suporte_Motor_Esteira_Rolante_SOE.catPART

Quantidade: 1 unidade

Aro de fixação do motor de passo da esteira. É responsável por auxiliar a fixação do motor em conjunto com o `Suporte_Motor_Esteira_Rolante_SOE`. As duas peças trabalham juntas para manter a fixação do motor de passo.

`Suporte_Parafuso_Esteira_Rolante_SOE.catPART`

Quantidade: 3 unidades

Suporte do parafuso que faz o alinhamento do rolo da esteira. O parafuso é fixado ao suporte e cria um ponto de eixo para o rolamento do rolo. É utilizado tanto para o rolo motriz (associado ao motor de passo) quanto para o rolo bobo (utilizando dois suportes para criar os dois pontos de eixo do rolo).

`Tampa_Rolo_Motor_Esteira_Rolante_SOE.catPART`

Quantidade: 1 unidade

Tampa do rolo motriz, que associa o eixo do motor de passo à estrutura do rolo. Em conjunto com a `Tampa_Rolo_Rolamento_Esteira_Rolante_SOE` e um cano de PVC de 50 mm, forma o rolo motriz da esteira.

`Tampa_Rolo_Rolamento_esteira_rolante_SOE.catPART`

Quantidade: 3 unidades

Tampa do rolo bobo, que associa o eixo do parafuso à estrutura do rolo. O rolo motriz utiliza uma unidade desta peça, enquanto o rolo bobo utiliza duas. A fixação do rolo bobo emprega dois `Suporte_Parafuso_Esteira_Rolante_SOE` em conjunto com dois rolamentos 608 que se apoiam nos parafusos fixados aos suportes.

`Suporte_mesa_Rolante_SOE.catPART`

Quantidade: 6 unidades

Suporte para a chapa de PVC que forma a mesa da esteira. Um conjunto dessas peças forma a base da mesa por onde corre o tapete, guiado pelo motor de passo a partir do rolo motriz e do rolo bobo.

`Lateral_esq_rampa_esteira_rolante_SOE.catPART`

Quantidade: 1 unidade

Lateral esquerda com alojamento para encaixe de uma chapa de PVC que forma a rampa da saída B da esteira.

Lateral_dir_rampa_esteira_rolante_SOE.catPART

Quantidade: 1 unidade

Lateral direita com alojamento para encaixe de uma chapa de PVC que forma a rampa da saída B da esteira.

Peças do Subsistema de Sensoriamento, Atuação e Eletrônica

Autor: Felipe de Castro

Cancela_servo.catPART

Quantidade: 1 unidade

Estrutura fixada ao eixo do servo motor responsável por guiar o objeto sobre a esteira para a saída B.

Suporte_Servo.catPART

Quantidade: 1 unidade

Peça de fixação do servo motor. Este suporte é colado sobre a mesa da esteira e posiciona o servo motor na altura e orientação necessárias para suprir as demandas do escopo do projeto.

Suporte_STlink.catPART

Quantidade: 1 unidade

Suporte para fixação do ST-Link (programador e debugger do STM32) no corpo da esteira.

Suporte_laser_c_ajuste_focal.catPART

Quantidade: 3 unidades

Suporte do laser com ajuste focal para o sensor laser. Serve para alinhar o laser ao LDR e deve ser fixado na mesa da esteira com bom alinhamento entre os dois dispositivos. Cada sensor disposto na mesa da esteira possui um suporte de laser.

Suporte_LDR.catPART

Quantidade: 3 unidades

Suporte do LDR. Serve para alinhar o LDR ao laser e deve ser fixado na mesa da esteira com bom alinhamento entre os dois dispositivos. Cada sensor na mesa possui um suporte de LDR.

Suporte_laser_Rampa.catPART

Quantidade: 1 unidade

Suporte do laser para a rampa. É colado à lateral da rampa e cria um ponto de fixação por parafuso que prende o conjunto à base da esteira. Deve ser colado alinhado ao suporte do LDR da rampa.

Suporte_LDR_sensor_Rampa.catPART

Quantidade: 1 unidade

Suporte do LDR para a rampa. É colado à lateral da rampa e cria um ponto de fixação por parafuso que prende o conjunto à base da esteira. Deve ser colado alinhado ao suporte do laser da rampa.

Suporte_flash_esteira.catPART

Quantidade: 2 unidades

Suporte para a fita de LED que ilumina o objeto na região de leitura do código QR.

Suporte_poste_camera.catPART

Quantidade: 2 unidades

Suporte plástico que une os dois suportes do flash e forma a estrutura da câmera, fechando a região de leitura da esteira. São utilizadas duas peças para separar igualmente os dois postes do flash.

Quantidade: 1 unidade

Ponto de fixação da câmera. Deve ser colado sobre os suportes que formam a região de leitura de forma que a câmera fique alinhada com o sensor laser onde o objeto é posicionado sobre o tapete da esteira.

Case de fixação do Raspberry Pi 3b

Autor: Nilo - Usuário do MakerWorld

O case de fixação do RBPi3B utilizado foi obtido através da plataforma Open Source *MakerWorld*, que é um site onde usuários da comunidade de impressão 3D compartilham seus próprios desenhos de forma aberta sem fim comercial.

A case foi desenhada inicialmente para o RBPi4 e após impressa foi modificada para abrir os alojamentos das portas USB e HDMI.

O case original pode ser encontrado no seguinte link <https://makerworld.com/pt/models/1500557-case-for-raspberry-pi-4-with-40mm-fan?from=search#profileId-1569716>

1. Contexto de Aplicação

Sistemas de logística interna em centros de distribuição utilizam linhas automatizadas de transporte e separação de itens para preparação de pedidos. Esses sistemas são compostos por módulos de esteiras interconectados, capazes de identificar e direcionar produtos conforme seu destino.

2. Definição do Problema

Dado um fluxo contínuo de itens previamente identificados, é necessário desenvolver um módulo de esteira automatizado capaz de:

- Permitir a identificação de cada item;
- Determinar o destino de cada item;
- Direcioná-lo corretamente dentro de uma linha de separação modular.

3. Objetivo do Projeto

Desenvolver um sistema embarcado para controle de um módulo de esteira automatizada com capacidade de identificação e desvio de itens de acordo com o destino previamente associado ao objeto, com ênfase na integração entre controle embarcado e processamento em sistema Linux embarcado.

Objetivos Específicos:

- Implementar controle de movimentação da esteira;
- Desenvolver mecanismo de desvio de itens;
- Integrar comunicação com sistema externo;
- Validar o funcionamento do sistema experimentalmente.

4. Escopo do Sistema

O sistema consiste em um módulo de esteira com:

- 1 entrada de itens;
- 2 saídas (desvio local ou encaminhamento para próximo módulo).

Capaz de operar com objetos que atendam às seguintes restrições:

- Massa máxima: 250 g;
- Volume máximo: 125 cm³;
- Altura: entre 2 cm e 5 cm.

5. Funcionalidades do Sistema

O sistema embarcado deverá:

- 1) Detectar a presença de objetos na esteira;
- 2) Controlar o deslocamento do item até a zona de leitura;
- 3) Solicitar ao servidor, via interface de comunicação, a identificação do item;
- 4) Receber do servidor a decisão de destino;
- 5) Executar o desvio do item para uma das saídas.

6. Interface com o Servidor (ex.: Raspberry Pi)

O módulo deverá possuir interface de comunicação com um servidor que será responsável por:

- Captura de imagem do item na zona de leitura;
- Processamento e decodificação do código QR;
- Determinação do destino do item;
- Envio de comandos ao módulo da esteira;
- Monitoramento do estado do sistema;
- Registro de eventos de operação.

7. Limitações do Projeto

Este projeto não contempla:

- Integração com sistemas reais de estoque;
- Implementação de múltiplos módulos interconectados;
- Processamento de múltiplos itens simultaneamente;
- Sistemas avançados de visão computacional.

8. Resultados Esperados

- Protótipo funcional de um módulo de esteira automatizado validado experimentalmente;
- Sistema embarcado operando integrado a um ambiente Linux embarcado;
- Capacidade de identificação e separação de itens conforme o destino definido;
- Documentação técnica completa do sistema.

9. Aplicação

O sistema proposto se aplica a processos de automação logística em centros de distribuição, podendo ser utilizado como base para sistemas modulares escaláveis de separação de pedidos. A arquitetura proposta segue o modelo de processamento distribuído, separando tarefas de tempo real e processamento de alto nível.

